

Sistema de Banca por Internet - BP

Documento de Arquitectura de Solución

Julio 2026

Contenido

1. Resumen.....	3
Características Principales	3
Arquitectura de Tres Capas según Enunciado	4
Patrones de Diseño Implementados	4
Decisiones Arquitectónicas Clave (Justificadas)	6
Cumplimiento Normativo	6
Indicadores Clave	7
2. Requerimientos del Sistema	8
2.1 Requerimientos Funcionales	8
2.2 Requerimientos No Funcionales	8
3. Consideraciones Normativas y de Cumplimiento	9
3.1 Regulaciones Aplicables.....	9
4. Decisiones Arquitectónicas	11
4.1 Elección de Framework para Aplicación Móvil	11
4.2 Elección de Framework para SPA.....	12
4.3 Arquitectura de Microservicios	13
4.4 Selección de API Gateway.....	13
4.5 Base de Datos.....	14
4.6 Proveedor Cloud.....	15
4.7 Solución de Identidad y OAuth 2.1 / OpenID Connect (OIDC)	15
4.8 Flujo OAuth 2.1 / OpenID Connect (OIDC) Recomendado	16
4.8.1 Justificación de flujo authorization code + PKCE.....	16
4.9 Reconocimiento Facial para Onboarding	17
4.10 Autenticación Biométrica Post-Onboarding.....	17
5. Diagrama de Contexto (C4 - Nivel 1).....	18
Actores y Sistemas	18

Flujos Principales	19
6. Diagrama de Contenedores (C4 - Nivel 2)	20
.....	20
Contenedores del Sistema	21
7. Diagrama de Componentes (C4 - Nivel 3).....	27
7.1 Transaction Service - Componentes Detallados	28
7.2 Customer Service - Componentes Detallados.....	29
7.3 Notification Service - Componentes Detallados	30
7.4 Fraud Detection Service - Componentes Detallados	31
8. Arquitectura de Autenticación y Autorización	31
8.1 Flujo de Autenticación - SPA Web.....	31
8.2 Flujo de Autenticación - Aplicación Móvil.....	33
8.3 Flujo de Onboarding con Reconocimiento Facial.....	34
8.4 Multi-Factor Authentication (MFA)	35
9. Arquitectura de Integración y API Gateway	36
9.1 Capa de API Gateway	36
9.2 Patrones de Integración.....	37
9.3 Protocolos de Comunicación.....	39
10. Arquitectura de Datos y Auditoría	39
10.1 Estrategia de Persistencia Poliglota.....	39
10.2 Patrón Event Sourcing para Auditoría.....	43
10.3 Mecanismo de Persistencia para Clientes Frecuentes	44
11. Alta Disponibilidad y Recuperación ante Desastres.....	45
11.1 Objetivos de Disponibilidad.....	45
11.2 Arquitectura Multi-Región	45
11.3 Estrategia de Failover.....	46
11.4 Auto-Scaling y Self-Healing.....	47
11.5 Backup y Recuperación	48
12. Seguridad y Monitoreo	49
12.1 Modelo de Seguridad en Capas.....	49
12.2 Gestión de Secretos	51
12.3 Arquitectura de Monitoreo y Observabilidad	52
12.4 Security Monitoring (SIEM).....	54
13. Estrategia de Cloud y Costos.....	54

13.1 Estimación de Costos Mensual (Producción).....	54
13.2 Estrategias de Optimización de Costos.....	56
13.3 FinOps - Gobernanza de Costos.....	57
14. Conclusiones.....	57
14.1 Resumen de Arquitectura.....	57
14.2 Roadmap de Implementación.....	58
14.3 Riesgos y Mitigaciones.....	59
14.4 Métricas de Éxito.....	60
14.5 Siguiendo Pasos.....	60
Anexos.....	61
Anexo A: Tecnologías y Versiones.....	61
Anexo B: Referencias y Estándares.....	61
Anexo C: Glosario.....	62

1. Resumen

El presente documento describe la arquitectura de solución para el sistema de banca por internet de BP Bank. El sistema permite a los clientes realizar operaciones bancarias de manera segura, incluyendo consultas de movimientos, transferencias entre cuentas propias y pagos interbancarios.

Características Principales

- Frontend Multi-plataforma: SPA web y aplicación móvil multiplataforma
- Autenticación Robusta: OAuth 2.1 / OpenID Connect (OIDC) con MFA y biometría
- Arquitectura de Microservicios: Desacoplada, escalable y mantenible
- Integración con Core Bancario: A través de capa de integración robusta
- Notificaciones Multi-canal: Email, SMS y Push notifications mediante Firebase Cloud Messaging (FCM)
- Base de Auditoría Completa: Amazon DocumentDB (compatible con MongoDB) con Event Sourcing, retención de 7 años
- Mecanismo de Persistencia para Clientes Frecuentes: Patrón Caché Warming (Predictive Caching)
- Alta Disponibilidad: 99.95% uptime con multi-región
- Cumplimiento Normativo: PCI-DSS, Ley Orgánica de Protección de Datos Personales (LOPD) y regulaciones bancarias ecuatorianas

Arquitectura de Tres Capas según Enunciado

La solución implementa tres microservicios core según especificación, más servicios complementarios para funcionalidad completa:

MICROSERVICIOS:

- Customer Service: Gestión de clientes, agregación de datos desde Core y Sistema Detallado
- Account Service: Consulta de saldos, movimientos y productos bancarios
- Transaction Service: Procesamiento de transferencias propias e interbancarias

MICROSERVICIOS COMPLEMENTARIOS:

- Authentication Service (Keycloak): OAuth 2.1 / OpenID Connect (OIDC), MFA, gestión de sesiones
- Notification Service: Orquestación de notificaciones multi-canal
- Onboarding Service: Registro KYC con reconocimiento facial
- Fraud Detection Service: Detección de fraude con Machine Learning
- Audit Service: Auditoría inmutable con Event Sourcing
- Core Integration Service: Anticorruption Layer para sistema legacy
- External Systems Integration: Integración con ACH, listas restrictivas

Patrones de Diseño Implementados

Esta arquitectura implementa múltiples patrones de diseño reconocidos por la industria:

PATRONES ARQUITECTÓNICOS:

- Microservices Architecture: Desacoplamiento y escalabilidad independiente
- API Gateway Pattern: AWS API Gateway (externo) + Kong (interno) en capas secuenciales
- Database per Service: Persistencia poliglota según necesidades
- Strangler Fig: Migración gradual del Core legacy

PATRONES DE RESILIENCIA:

- Circuit Breaker (Resilience4j): Protección contra cascade failures
- Saga Pattern (Orchestration): Transacciones distribuidas con compensación
- Retry with Exponential Backoff: Reintento inteligente de operaciones
- Bulkhead: Aislamiento de recursos por servicio

PATRONES DE DATOS:

- Caché-Aside: Reducción de latencia 70-80%
- Caché Warming (Predictive Caching): Mecanismo para clientes frecuentes (responde a enunciado)
- Event Sourcing: Auditoría inmutable, trazabilidad completa
- CQRS: Separación Command/Query en Transaction Service
- Outbox Pattern: Garantía de entrega de eventos
- Repository Pattern: Abstracción de persistencia

PATRONES DE INTEGRACIÓN:

- Anticorruption Layer: Protección contra cambios en Core legacy
- Adapter Pattern: CoreBankingAdapter para transformación SOAP↔REST
- Aggregator Pattern: Customer Service agrega datos de múltiples fuentes

PATRONES DE DISEÑO (DDD):

- Aggregate Pattern: TransferAggregate como raíz
- Domain Events: Comunicación desacoplada entre bounded contexts
- Specification Pattern: Validaciones de negocio componibles

Decisiones Arquitectónicas Clave (Justificadas)

Cada decisión tecnológica cuenta con 4-6 justificaciones detalladas en Sección 4:

- React Native: Maximiza reutilización de código con SPA React, ecosistema maduro para biometría
- React + TypeScript: Type safety crítico para operaciones financieras, ecosistema robusto
- Kubernetes (EKS): Orquestación de contenedores, auto-scaling, self-healing
- AWS API Gateway + Kong: Doble gateway en capas (externo/interno), defensa en profundidad
- PostgreSQL + Redis + Amazon DocumentDB (compatible con MongoDB): Persistencia poliglota según patrones de acceso
- AWS: Certificaciones financieras, presencia regional LATAM, servicios gestionados
- Keycloak: OAuth 2.1 / OpenID Connect (OIDC)/OIDC completo, open source, sin licensing fees
- Authorization Code + PKCE: Estándar OAuth 2.1 / OpenID Connect (OIDC) Security Best Practices (RFC 8252)
- AWS Rekognition: Liveness detection, Alta precisión para validación biométrica y detección de vida (liveness detection), cumplimiento KYC digital
- FIDO2/WebAuthn: Biometría local, passwordless, hardware-backed security

Cumplimiento Normativo

REGULACIONES IMPLEMENTADAS:

- Ley Orgánica de Protección de Datos Personales (LOPD)
- PCI-DSS v4.0 (Payment Card Industry)
- Normativa de Seguridad de la Información y Gestión de Riesgos emitida por la Superintendencia de Bancos del Ecuador
- Sistema de Prevención de Lavado de Activos y Financiamiento de Delitos conforme normativa ecuatoriana- ISO 27001 (Seguridad de la Información)

CONTROLES DE SEGURIDAD:

- Defensa en profundidad: 4 capas de seguridad (Edge, Network, Application, Data)
- Encriptación end-to-end: TLS 1.3 en tránsito, AES-256 en reposo
- Tokenización: Datos sensibles tokenizados con HashiCorp Vault
- Auditoría inmutable: 100% transacciones auditadas, retención 7 años
- MFA obligatorio para transferencias superiores a USD 1.000 y operaciones de alto riesgo

Indicadores Clave

DISPONIBILIDAD Y PERFORMANCE:

- SLA: 99.95% uptime (4.38 horas downtime/año máximo)
- Latencia consultas: <200ms (P95)
- Latencia transacciones: <500ms (P95)
- Throughput: 10,000 TPS en picos
- RPO: < 5 minutos (pérdida máxima de datos)
- RTO: <4 horas (tiempo máximo de recuperación)

ESCALABILIDAD:

- Auto-scaling horizontal: HPA en Kubernetes basado en CPU, memoria y métricas custom
- Multi-región: Active-Active para lecturas, Active-Passive para escrituras
- Caché hit rate: >80% (reduce carga DB en 70-80%)

COSTOS:

- Estimación mensual: \$7,100 USD (optimizado con Reserved Instances)
- Estimación anual: \$85,200 USD
- Optimización: 23% reducción vs. precio on-demand

2. Requerimientos del Sistema

2.1 Requerimientos Funcionales

1. **Gestión de Clientes**
 - Consulta de información básica del cliente desde Core bancario
 - Consulta detallada desde sistema complementario
 - Onboarding de nuevos clientes con reconocimiento facial
2. **Operaciones Bancarias**
 - Consulta de histórico de movimientos
 - Transferencias entre cuentas propias
 - Transferencias interbancarias
 - Pagos de servicios
3. **Notificaciones**
 - Notificación por email de transacciones
 - Notificación por SMS para operaciones críticas
 - Push notifications mediante Firebase Cloud Messaging (FCM) en aplicación móvil
 - Alertas de seguridad
4. **Autenticación y Seguridad**
 - Login con usuario y contraseña
 - Autenticación biométrica (huella digital, reconocimiento facial)
 - Multi-factor authentication (MFA) implementado mediante TOTP y WebAuthn/FIDO2.
 - Token-based authentication con OAuth 2.1 / OpenID Connect (OIDC)

2.2 Requerimientos No Funcionales

5. **Disponibilidad:** 99.95% uptime (máximo 4.38 horas de downtime al año)
6. **Performance:**
 - Latencia < 200ms para operaciones de consulta
 - Latencia < 500ms para operaciones transaccionales
 - Throughput objetivo: hasta 10.000 TPS en escenarios pico, sujeto a validación mediante pruebas de carga, dimensionamiento definitivo de infraestructura y patrones reales de uso.
7. **Escalabilidad:** Auto-scaling horizontal basado en métricas

8. **Seguridad:**
 - Encriptación en tránsito (TLS 1.3)
 - Encriptación en reposo (AES-256)
 - Tokenización de datos sensibles
 - WAF para protección de aplicaciones web
 9. **Recuperación ante Desastres:**
 - RPO (Recovery Point Objective): < 5 minutos
 - RTO (Recovery Time Objective): < 4 horas
 10. **Auditoría:** 100% de transacciones auditadas con retención de 7 años
-

3. Consideraciones Normativas y de Cumplimiento

3.1 Regulaciones Aplicables

3.1.1 Ley Orgánica de Protección de Datos Personales (LOPDP)

Consideraciones:

- Implementar políticas de privacidad claras y accesibles
- Obtener consentimiento explícito para tratamiento de datos biométricos
- Garantizar Derechos de acceso, rectificación, actualización, eliminación, oposición y portabilidad de datos conforme LOPDP
- Notificar brechas de seguridad en máximo 72 horas
- Implementar Privacy by Design en toda la arquitectura

Implementación en Arquitectura:

- Microservicio de Gestión de Consentimientos
- Encriptación de datos personales en base de datos
- Logs de auditoría para acceso a datos personales
- API para ejercicio de derechos de acceso, rectificación, actualización, eliminación, oposición y portabilidad de datos conforme LOPDP

3.1.2 PCI-DSS (Payment Card Industry Data Security Standard)

Consideraciones:

- Nunca almacenar CVV o datos completos de tarjetas
- Tokenización de números de cuenta
- Segmentación de red (DMZ, zona segura)
- Controles de acceso estrictos basados en roles
- Monitoreo y logging de accesos a datos de tarjetas
- Escaneo de vulnerabilidades trimestral - Penetration testing anual

Implementación en Arquitectura:

- Tokenization Service para enmascarar datos sensibles
- Network segmentation con Security Groups y NACLs
- Vault (HashiCorp Vault) para gestión de secretos
- IDS/IPS para detección de intrusiones

3.1.3 Normativa de Seguridad de la Información y Gestión de Riesgos emitida por la Superintendencia de Bancos del Ecuador

Consideraciones:

- Gestión de riesgos de ciberseguridad
- Controles de seguridad en canales electrónicos
- Autenticación reforzada en transacciones críticas
- Monitoreo continuo de transacciones sospechosas

Implementación en Arquitectura:

- Sistema de detección de fraude con ML
- MFA obligatorio para transferencias superiores a USD 1.000 y operaciones de alto riesgo
- Rate limiting y throttling en APIs
- Análisis de comportamiento de usuarios

3.1.4 Sistema de Prevención de Lavado de Activos y Financiamiento de Delitos conforme normativa ecuatoriana

Consideraciones:

- Identificación y validación de clientes (KYC)
- Monitoreo de transacciones inusuales
- Reportes a UAFE cuando aplique
- Listas restrictivas (OFAC, ONU, etc.)

Implementación en Arquitectura:

- Servicio de validación contra listas restrictivas
- Motor de reglas para detección de operaciones sospechosas
- Integración con sistema de compliance
- Auditoría inmutable de transacciones

3.1.5 ISO 27001 - Seguridad de la Información

Consideraciones:

- SGSI (Sistema de Gestión de Seguridad de la Información)
- Gestión de riesgos de seguridad
- Controles de acceso y cifrado
- Gestión de incidentes de seguridad
- Continuidad del negocio

4. Decisiones Arquitectónicas

4.1 Elección de Framework para Aplicación Móvil

Opciones Evaluadas:

Opción 1: React Native

Justificaciones:

1. **Reutilización de código:** Comparte lógica de negocio con la SPA (React), reduciendo tiempo de desarrollo en 40-60%

2. **Ecosistema maduro:** Amplia comunidad, librerías probadas en producción para autenticación biométrica (react-native-biometrics), cámara para reconocimiento facial (react-native-vision-camera), y notificaciones push
3. **Performance cercano a nativo:** Con Hermes engine y TurboModules, el rendimiento es comparable a apps nativas
4. **Hot reload:** Acelera el desarrollo y debugging
5. **Mantenibilidad:** Un equipo de desarrolladores JavaScript puede mantener web y móvil

Opción 2: Flutter

Justificaciones:

6. **Performance superior:** Compilación a código nativo sin bridge, mejor para animaciones complejas
7. **UI consistente:** Widgets propios que se ven idénticos en iOS y Android
8. **Dart language:** Tipado fuerte, mejor para equipos grandes

Decisión Final: React Native

Razones:

- Maximizar reutilización de código con la SPA React
- Equipo de desarrollo ya familiarizado con JavaScript/TypeScript
- Librerías maduras para biometría (react-native-touch-id, react-native-face-id) - Integración nativa con OAuth2 (react-native-app-auth)
- Mejor ecosistema para integraciones bancarias específicas

4.2 Elección de Framework para SPA

Decisión: React + TypeScript

Justificaciones:

1. **Ecosistema robusto:** Librerías empresariales como Material-UI, Ant Design con componentes de seguridad incluidos
2. **TypeScript:** Type safety crítico para operaciones financieras, reduce bugs en producción en 15-20%
3. **Code splitting:** Optimización de carga, crítico para usuarios con conexiones lentas

4. **SSR capable:** Posibilidad de implementar Next.js para mejorar SEO y performance
5. **Testing:** Jest, React Testing Library, Cypress para testing robusto

Alternativa evaluada: Vue.js (descartada por menor ecosistema enterprise)

4.3 Arquitectura de Microservicios

Decisión: Microservicios en Kubernetes (AWS EKS)

Justificaciones:

1. **Desacoplamiento:** Permite que equipos trabajen independientemente en servicios específicos
2. **Escalabilidad independiente:** Cada servicio escala según su carga (ej: servicio de consultas escala más que transferencias)
3. **Resiliencia:** Fallo de un servicio no afecta a otros (Circuit Breaker pattern)
4. **Deployment independiente:** CI/CD por servicio, reduciendo riesgo de despliegues
5. **Tecnología heterogénea:** Permite usar el mejor stack para cada problema (Node.js para APIs, Python para ML anti-fraude)

Alternativa evaluada: Monolito modular (descartado por limitaciones de escalabilidad)

Service Mesh: No se incorpora una solución de Service Mesh (Istio o Linkerd) en la fase inicial debido a la complejidad operativa adicional que introduce. Las capacidades actuales de observabilidad, seguridad y resiliencia son cubiertas mediante Kong, OpenTelemetry, API Gateway y políticas de Kubernetes. Su adopción podrá evaluarse en fases posteriores cuando existan requerimientos avanzados de mTLS, traffic shaping o gestión granular del tráfico entre servicios.

4.4 Selección de API Gateway

Decisión: AWS API Gateway + Kong

Justificaciones:

1. AWS API Gateway (Externo):

- Integración nativa con AWS WAF para protección DDoS
- Rate limiting y throttling automático
- Integración con CloudWatch para observabilidad
- Soporte nativo para OAuth 2.1 / OpenID Connect (OIDC) y JWT

- Caché integrado para reducir latencia

2. Kong (Interno):

- API Gateway interno para routing, observabilidad y políticas de seguridad entre microservicios
- Plugins para autenticación, rate limiting, transformación
- Open source con versión enterprise para soporte
- Alto rendimiento (basado en Nginx)

Alternativas evaluadas:

Opción 1: AWS API Gateway solo (limitaciones en routing complejo interno)

Opción 2: NGINX+ (menos features out-of-the-box)

4.5 Base de Datos

Decisión: PostgreSQL (AWS RDS) + Redis (ElastiCaché) + Amazon DocumentDB (compatible con MongoDB)

Consideración sobre Event Sourcing: Amazon DocumentDB se utiliza como repositorio de auditoría y consulta histórica de eventos regulatorios. Los eventos de dominio son publicados mediante EventBridge y consumidos por servicios especializados. El objetivo principal es garantizar trazabilidad, auditoría e inmutabilidad para cumplimiento normativo, más que implementar un Event Store puro orientado a reconstrucción completa de estado.

Justificaciones:

PostgreSQL para datos transaccionales:

- **ACID compliance:** Crítico para operaciones financieras
- **Soporte JSONB:** Flexibilidad para datos semi-estructurados
- **Performance:** Excelente para queries complejas con índices
- **Replicación:** Multi-AZ para HA
- **Maduro y probado:** Usado por instituciones financieras globalmente

Redis para caché:

- **Ultra baja latencia:** < 1ms para datos en memoria
- **Patrón Caché-Aside:** Reduce carga en PostgreSQL en 70-80%
- **Session storage:** Almacenamiento de sesiones de usuario
- **Rate limiting:** Control de intentos de autenticación

Amazon DocumentDB (compatible con MongoDB) para auditoría:

- **Writes rápidos:** Optimizado para inserción masiva de logs
- **Schema flexible:** Auditoría de diferentes tipos de eventos
- **Time-series:** Consultas eficientes por rango de fechas
- **Retención:** Fácil implementación de TTL para cumplimiento

4.6 Proveedor Cloud

Decisión: Amazon Web Services (AWS)

Justificaciones:

1. **Madurez en servicios financieros:** AWS tiene certificaciones PCI-DSS, SOC 1/2/3, ISO 27001
2. **Presencia regional:** Múltiples regiones en Latinoamérica (São Paulo, etc.)
3. **Servicios gestionados:** RDS, EKS, ElastiCaché reducen overhead operacional
4. **Seguridad:** AWS KMS, Secrets Manager, GuardDuty para threat detection
5. **Costos:** Pricing competitivo con Reserved Instances y Savings Plans
6. **Ecosistema:** Amplio marketplace de soluciones financieras pre-certificadas

Alternativa evaluada: Azure (descartado por menor presencia regional en LATAM)

4.7 Solución de Identidad y OAuth 2.1 / OpenID Connect (OIDC)

Decisión: Keycloak (Open Source Identity Provider)

Justificaciones:

1. **OAuth 2.1 / OpenID Connect (OIDC) y OIDC:** Soporte completo de estándares
2. **Multi-factor:** MFA integrado (TOTP, SMS, Email)
3. **Social login:** Integración con proveedores externos si se requiere
4. **Escalable:** Clusterización para alta disponibilidad
5. **Customizable:** Extensible con SPIs para integraciones específicas
6. **Costo:** Open source, sin licensing fees

Alternativas evaluadas:

Auth0: SaaS costoso (\$23k+/año para volumen esperado)

AWS Cognito: Limitaciones en customización de flujos

4.8 Flujo OAuth 2.1 / OpenID Connect (OIDC) Recomendado

Para SPA Web: Authorization Code Flow con PKCE

Justificaciones:

1. **Seguridad:** PKCE previene authorization code interception
2. **Estándar moderno:** Recomendación de OAuth 2.1 / OpenID Connect (OIDC) Security Best Practices (RFC 8252)
3. **No requiere client secret:** Seguro para aplicaciones públicas
4. **Refresh tokens:** Permite sesiones largas sin re-autenticación constante

Para Aplicación Móvil: Authorization Code Flow con PKCE + AppAuth

Justificaciones:

1. **Sistema browser nativo:** Usa ASWebAuthenticationSession (iOS) o Chrome Custom Tabs (Android)
2. **Seguridad mejorada:** No expone credenciales a la app
3. **Biometría post-autenticación:** Refresh token protegido con biometría local
4. **Compliance:** Cumple con recomendaciones de FAPI (Financial API)

4.8.1 Justificación de flujo authorization code + PKCE

Alternativas evaluadas:

- Implicit Flow
- Client Credentials
- Resource Owner Password Credentials

Decisión:

Authorization Code + PKCE.

Justificación:

1. Recomendado por OAuth 2.1.
2. Mitiga ataques de interceptación de código.
3. Adecuado para SPA y aplicaciones móviles.

5. Compatible con OpenID Connect.
6. Adoptado por entidades financieras y Open Banking.

4.9 Reconocimiento Facial para Onboarding

Decisión: AWS Rekognition + Liveness Detection

Justificaciones:

1. **Liveness detection:** Previene ataques con fotos o videos
2. **Face matching:** Compara selfie con documento de identidad
3. **Compliance:** Cumple con regulaciones de KYC digital
4. **Escalabilidad:** Serverless, pago por uso
5. Alta precisión para validación biométrica y detección de vida (liveness detection)

Alternativas evaluadas: - **Face++ (Megvii):** Preocupaciones de privacidad de datos en China - **Azure Face API:** Menor precisión en pruebas de liveness

Integración: - Cliente captura selfie + documento ID - Backend valida con Rekognition
- Se almacena embedding facial (no imagen) para futuras validaciones - Gestión de consentimientos (LOPDP): Usuario consiente explícitamente

4.10 Autenticación Biométrica Post-Onboarding

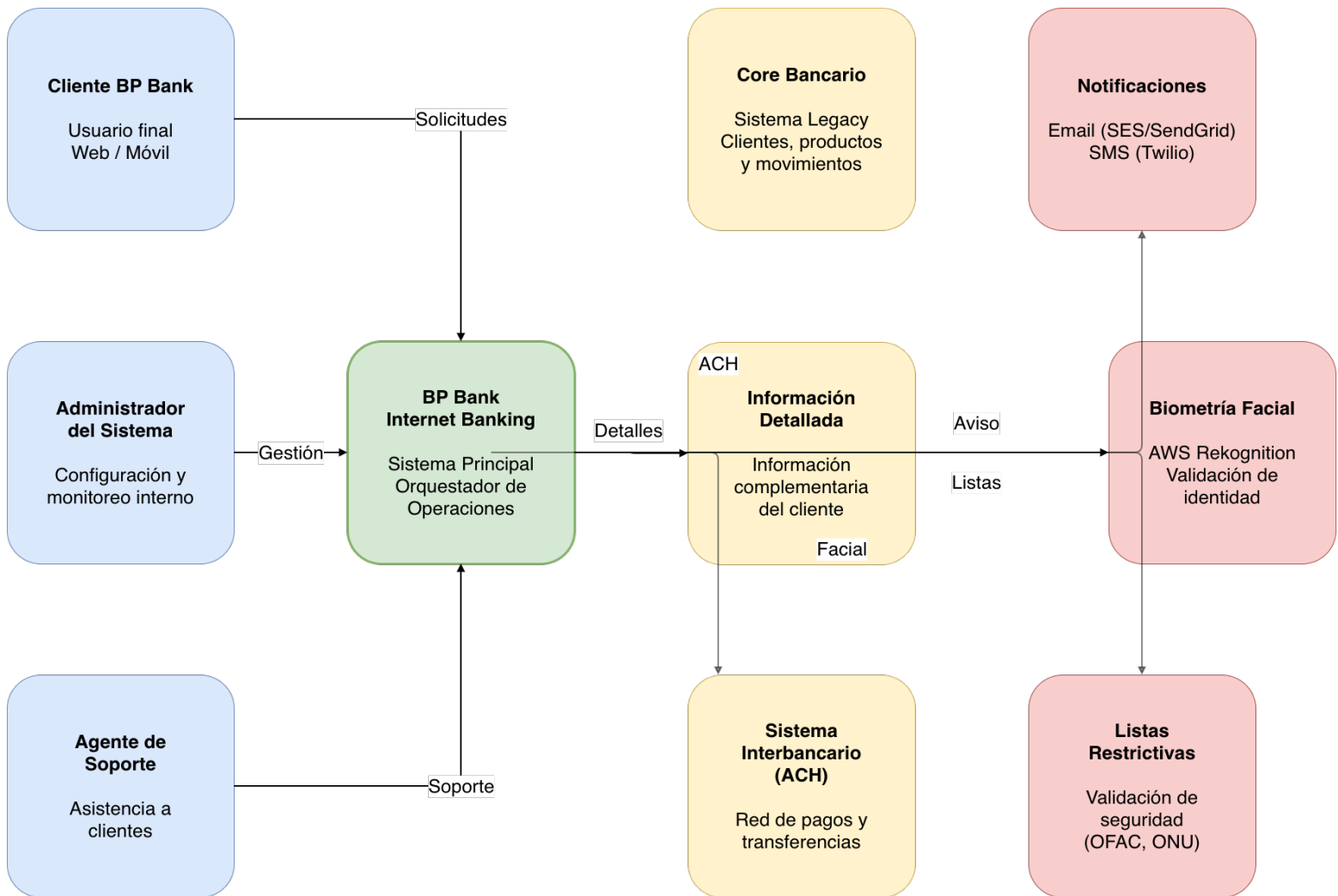
Decisión: FIDO2/WebAuthn + Biometría Local

Justificaciones:

1. **Passwordless:** Reduce riesgo de phishing y credential stuffing
2. **Biometría local:** Huella/Face ID no sale del dispositivo
3. **Estándar W3C:** Soporte nativo en navegadores modernos
4. **Hardware-backed:** Usa Secure Enclave (iOS) o TEE (Android)

Implementación: - Usuario registra dispositivo con biometría - Se genera par de claves pública/privada en hardware - Clave privada nunca sale del dispositivo - Backend valida firma con clave pública

5. Diagrama de Contexto (C4 - Nivel 1)



Actores y Sistemas

Actores:

Cliente BP Bank: Usuario final que accede desde web o móvil para realizar operaciones bancarias.

Administrador del Sistema: Personal interno que configura y monitorea el sistema.

Agente de Soporte: Personal que asiste a clientes con problemas

Sistemas Externos:

Core Bancario: Sistema legacy que contiene información básica de clientes, productos y movimientos

Sistema de Información Detallada: Sistema complementario con información extendida del cliente

Sistema Interbancario (ACH): Red de pagos para transferencias entre diferentes bancos

Servicio de Email (SES/SendGrid): Envío de notificaciones por correo electrónico

Servicio de SMS (Twilio): Envío de notificaciones por mensaje de texto

Proveedor de Reconocimiento Facial (AWS Rekognition): Validación biométrica en onboarding

Sistema de Listas Restrictivas: Validación de clientes contra listas OFAC, ONU, etc.

Sistema Principal:

BP Bank Internet Banking: Sistema de banca por internet que orquesta todas las operaciones

Flujos Principales

1. Consulta de información:

- Cliente accede al sistema
- Sistema obtiene datos del Core y Sistema de Información Detallada
- Sistema presenta información al cliente

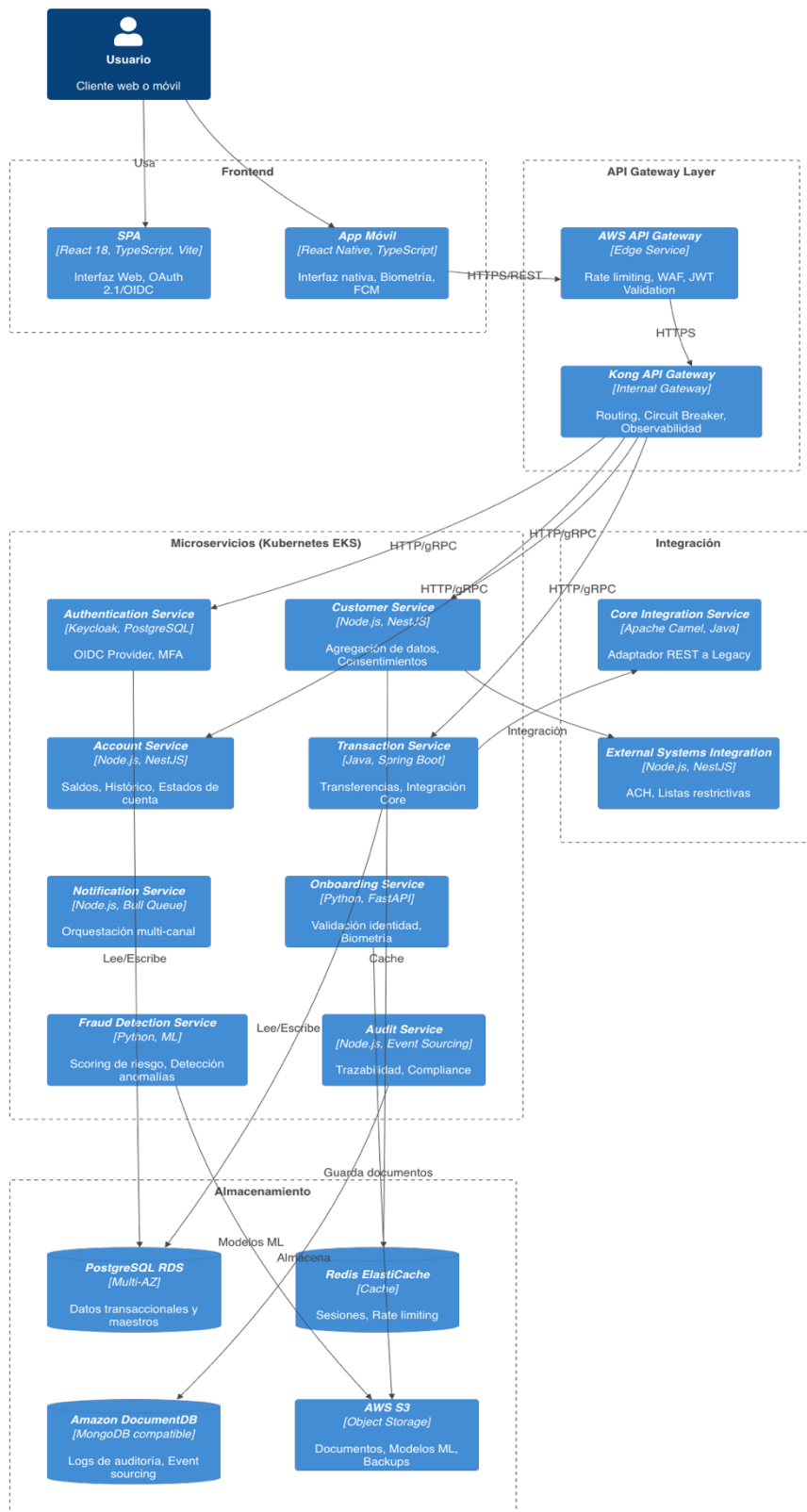
2. Transferencias:

- Cliente solicita transferencia
- Sistema valida en Core Bancario
- Si es interbancaria, se procesa vía ACH
- Sistema envía notificaciones vía Email/SMS/Push

3. Onboarding:

- Nuevo cliente inicia registro
 - Sistema captura biometría
 - Reconocimiento facial valida identidad
 - Sistema registra usuario en Core
-

6. Diagrama de Contenedores (C4 - Nivel 2)



Contenedores del Sistema

Frontend

1. Single Page Application (SPA):

Tecnología: React 18 + TypeScript + Vite.

Hosting: AWS CloudFront + S3.

Responsabilidades:

- Interfaz de usuario para clientes web
- Autenticación OAuth 2.1 / OpenID Connect (OIDC) con PKCE
- Responsive design para desktop y tablet.

Comunicación: HTTPS/REST con API Gateway

2. Aplicación Móvil

Tecnología: React Native + TypeScript.

Plataformas: iOS 14+ y Android 10+.

Responsabilidades:

- Interfaz nativa para smartphones
- Autenticación biométrica (Face ID, Touch ID)
- Push notifications mediante Firebase Cloud Messaging (FCM)
- Onboarding con captura de biometría.

Comunicación: HTTPS/REST con API Gateway

API Gateway Layer

3. AWS API Gateway (Edge):

Responsabilidades:

- Punto de entrada único para clientes externos
- Rate limiting y throttling (1000 req/min por usuario)
- Validación de JWT tokens
- Caché de respuestas (TTL 60s para consultas)
- WAF integration para protección

Comunicación: HTTPS hacia Kong Gateway interno

4. Kong API Gateway (Interno)

Responsabilidades:

- Routing interno entre microservicios
- Aplicación de políticas de seguridad
- Rate limiting interno
- Observabilidad y trazabilidad
- Circuit breaker pattern
- Request/Response transformation

Logging y métricas

- Routing inteligente
- Circuit breaker pattern
- Request/Response transformation

Comunicación: HTTP/gRPC con microservicios

Microservicios (Kubernetes EKS)

5. Authentication Service:

Tecnología: Keycloak 24.x + PostgreSQL

Responsabilidades:

- OAuth 2.1 / OpenID Connect (OIDC) / OIDC provider
- Multi-factor authentication
- Session management
- User federation con Core Bancario

Base de Datos: PostgreSQL RDS (Multi-AZ)

6. Customer Service:

Tecnología: Node.js + NestJS + TypeScript

Responsabilidades:

- Agregación de datos de cliente (Core + Sistema Detallado)
- Caché de perfiles de usuario

- Gestión de consentimientos (LOPD)

Base de Datos: PostgreSQL RDS + Redis ElastiCaché

- Registro de aceptación, revocación y trazabilidad de consentimientos del titular de datos

7. Account Service:

Tecnología: Node.js + NestJS

Responsabilidades:

- Consulta de saldos y productos
- Histórico de movimientos
- Estados de cuenta

Base de Datos: PostgreSQL RDS (Read Replicas)

8. Transaction Service:

Tecnología: Java + Spring Boot

Responsabilidades:

- Procesamiento de transferencias
- Validaciones de negocio
- Integración con Core para débitos/créditos
- Integración con ACH para interbancarias

Base de Datos: PostgreSQL RDS + Event Store

9. Notification Service:

Tecnología: Node.js + Bull Queue

Responsabilidades:

- Orquestación de notificaciones multi-canal
- Retry logic para entregas fallidas
- Templates de mensajes
- Tracking de entregas

Queue: AWS SQS

Proveedores: AWS SES, Twilio, FCM

10. Onboarding Service:

Tecnología: Python + FastAPI

Responsabilidades:

- Registro de nuevos clientes
- Validación de identidad con reconocimiento facial
- Validación de documentos
- Integración con listas restrictivas

Servicios: AWS Rekognition, Textract

11. Fraud Detection Service:

Tecnología: Python + Scikit-learn + TensorFlow

Responsabilidades:

- Análisis de patrones de transacciones
- Scoring de riesgo en tiempo real
- Detección de anomalías con ML
- Integración con motor de reglas

Base de Datos: Redis (caché) + S3 (modelos)

12. Audit Service:

Tecnología: Node.js + Event Sourcing

Responsabilidades

- Registro inmutable de todas las transacciones
- Trazabilidad completa de operaciones
- Compliance reporting
- Retención por 7 años

Base de Datos: Amazon DocumentDB (compatible con MongoDB)

Bases de Datos y Almacenamiento

13. PostgreSQL RDS (Multi-AZ):

- Datos transaccionales y maestros
- Backups automáticos diarios

- Snapshots antes de deployments

14. Redis ElastiCache

- Caché de sesiones de usuario
- Caché de consultas frecuentes
- Rate limiting counters

15. Amazon DocumentDB (compatible con MongoDB)

- Logs de auditoría
- Event sourcing
- Time-series data

16. AWS S3

- Documentos de clientes
- Modelos de ML
- Backups de bases de datos

Integración con Sistemas Externos

17. Core Integration Service

Tecnología: Apache Camel + Java

Responsabilidades:

- Adaptador entre APIs REST modernas y Core legacy
- Transformación de mensajes
- Manejo de timeouts y reintentos
- Circuit breaker para proteger Core

Protocolos: REST → SOAP/MQ según Core

18. External Systems Integration Service

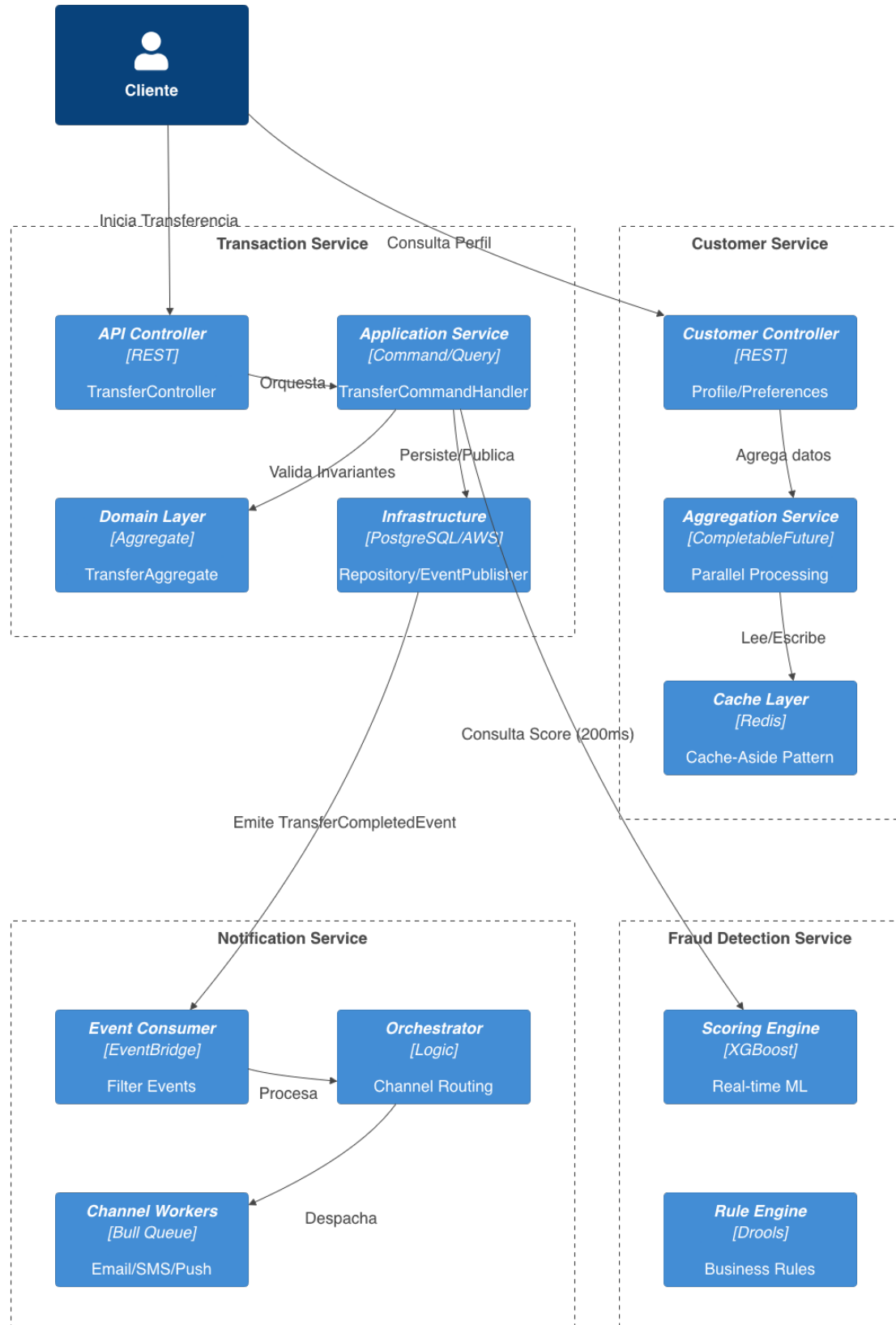
Tecnología: Node.js + NestJS

Responsabilidades:

- Integración con Sistema de Información Detallada
- Integración con ACH para interbancarias
- Integración con listas restrictivas

Protocolos: REST, SOAP, webhook

7. Diagrama de Componentes (C4 - Nivel 3)



7.1 Transaction Service - Componentes Detallados

Arquitectura interna basada en Domain-Driven Design (DDD) y CQRS

Componentes

1. API Controller Layer - TransferController: Endpoints REST para transferencias -
POST /api/v1/transfers/own - Transferencias entre cuentas propias - POST
/api/v1/transfers/interbank - Transferencias interbancarias - GET
/api/v1/transfers/{id} - Consulta de transferencia - GET
/api/v1/transfers/history - Histórico de transferencias

- **Security Interceptor:**
 - Validación de JWT token
 - Extracción de claims (userId, roles)
 - Rate limiting por usuario (100 transfers/día)

2. Application Service Layer - TransferCommandHandler: - Implementa patrón Command Pattern - Validaciones de negocio previas - Orquestación de flujo transaccional - Publicación de eventos

- **TransferQueryHandler:**
 - Implementa patrón Query (CQRS)
 - Optimizado para lecturas
 - Caché de consultas frecuentes

3. Domain Layer - TransferAggregate: - Entidad raíz del agregado - Invariantes de negocio - Emisión de domain events

- **Business Rules Engine:**
 - Validación de límites de transferencia
 - Horarios permitidos
 - Validación de cuentas origen/destino
 - Scoring anti-fraude

4. Infrastructure Layer - TransferRepository: - Persistencia en PostgreSQL - Implementa pattern Unit of Work - Transacciones ACID

- **CoreBankingAdapter:**
 - Anticorruption Layer para Core legacy
 - Circuit breaker (Resilience4j)
 - Timeout: 5 segundos
 - Retry: 3 intentos con backoff exponencial
- **EventPublisher:**
 - Publica eventos a AWS EventBridge

- Garantiza procesamiento idempotente mediante Outbox Pattern y deduplicación de eventos.
- Outbox pattern para consistencia

5. Cross-Cutting Concerns - Logging (SLF4J + Logback): Structured JSON logs a CloudWatch - **Metrics (Micrometer):** Métricas de negocio y técnicas a Prometheus - **Tracing (OpenTelemetry):** Distributed tracing a AWS X-Ray - **Exception Handling:** Global exception handler con códigos de error estandarizados

Flujo de Transferencia (Saga Pattern)

1. Cliente → API Gateway → Transaction Service
2. TransferController recibe request
3. Security Interceptor valida JWT y rate limits
4. TransferCommandHandler orquesta:
 - a. Validación con FraudDetectionService (sync)
 - b. Consulta saldo en AccountService (sync)
 - c. Validación business rules
 - d. Inicio de Saga distribuida:
 - Comando: DebitAccountCommand → Core
 - Evento: AccountDebited
 - Comando: CreditAccountCommand → Core/ACH
 - Evento: AccountCredited
 - Comando: SendNotificationCommand → Notification Service
5. TransferRepository persiste transacción
6. EventPublisher emite TransferCompletedEvent
7. AuditService consume evento (async)
8. Response al cliente

Compensación en caso de fallo: - Si falla crédito → Reversa automática de débito - Patrón Saga con compensating transactions - Estado de transacción: PENDING → COMPLETED | FAILED | COMPENSATED

7.2 Customer Service - Componentes Detallados

Componentes

1. API Controller Layer - CustomerController: - GET /api/v1/customers/profile - Perfil del cliente - GET /api/v1/customers/detailed - Información detallada - PATCH /api/v1/customers/preferences - Actualizar preferencias

2. Application Service Layer - CustomerAggregationService: - Implementa patrón **Aggregator Pattern** - Consulta paralela a múltiples fuentes - Timeout independiente por fuente - Fallback a datos en caché si fuente falla

3. Caché Layer (Redis) - Caché-Aside Pattern: - TTL: 5 minutos para datos básicos - TTL: 1 hora para datos detallados - Invalidación proactiva en updates - Warm-up de caché para clientes frecuentes

4. Integration Layer - CoreBankingClient: - Consulta datos básicos del Core - Circuit breaker: 50% error rate → OPEN - Bulkhead: Max 20 conexiones concurrentes

- **DetailedInfoClient:**
 - Consulta sistema de información detallada
 - Timeout: 3 segundos
 - Fallback: Devuelve solo datos básicos

Flujo de Consulta con Agregación

1. Cliente solicita perfil completo
2. CustomerAggregationService ejecuta en paralelo:
 - Thread 1: Caché lookup (Redis)
 - Si caché MISS:
 - Thread 2: CoreBankingClient.getBasicInfo()
 - Thread 3: DetailedInfoClient.getDetailedInfo()
 - CompletableFuture.allOf() espera ambas
3. Agregación de respuestas
4. Almacenamiento en caché
5. Response al cliente

Métricas: - Caché Hit Rate: Target > 80% - P95 latency: < 150ms con caché hit - P95 latency: < 500ms con caché miss

7.3 Notification Service - Componentes Detallados

Componentes

1. **Event Consumer** - Consume eventos de AWS EventBridge - Filtros por tipo de evento: - TransferCompleted - LoginAttemptFailed - PasswordChanged - LargeTransaction (> \$1000 USD)
2. **Notification Orchestrator** - Determina canales según: - Preferencias de usuario - Tipo de evento (crítico → SMS + Email + Push) - Horario (no SMS entre 10pm-8am) - Enqueue mensajes en colas específicas
3. **Channel Workers (Bull Queue) - EmailWorker:** - Procesa cola de emails - Templates con Handlebars - Envío vía AWS SES - Tracking de opens/clicks
 - **SMSWorker:**
 - Procesa cola de SMS
 - Twilio API
 - Validación de números internacionales
 - Retry: 3 intentos
 - **PushWorker:**
 - Procesa cola de Push notifications mediante Firebase Cloud Messaging (FCM)
 - Apple Push Notification Service (APNS)
 - Gestión de device tokens

4. Delivery Tracking - Estado: QUEUED → SENT → DELIVERED | FAILED - Persistencia en Amazon DocumentDB (compatible con MongoDB) - Webhooks de proveedores para confirmación

Patrón de Resiliencia

- **Dead Letter Queue:** Mensajes fallados después de 3 intentos
- **Rate Limiting:**
 - Email: 100/segundo (límite SES)
 - SMS: 10/segundo (límite Twilio)
 - Push: 500/segundo
- **Backpressure:** Cola con max 100k mensajes

7.4 Fraud Detection Service - Componentes Detallados

Componentes

1. Real-time Scoring Engine - Modelo de ML entrenado con XGBoost - Features: - Monto de transacción vs. promedio histórico - Hora del día - Ubicación geográfica (IP) - Dispositivo - Velocidad de transacciones - Output: Score 0-100 (>80 = alto riesgo)

2. Rule Engine (Drools) - Reglas de negocio configurables: - Transferencia > \$3000 USD → MFA obligatorio - 3+ transferencias en 5 minutos → Bloqueo temporal - Primera transferencia a nuevo beneficiario → Challenge adicional - Transacción desde nuevo dispositivo → SMS OTP

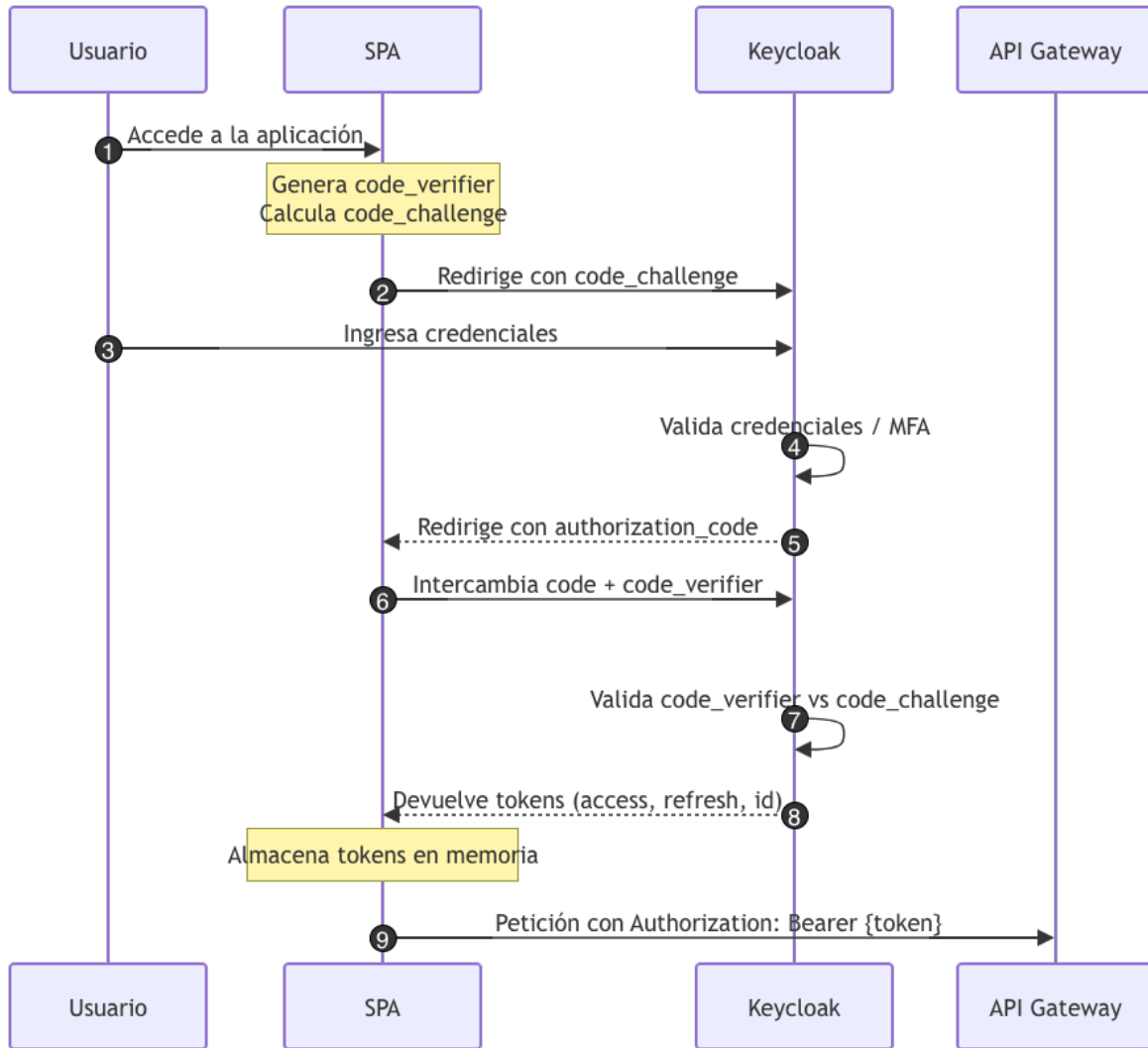
3. Behavioral Analysis - Perfil de comportamiento por usuario - Detección de anomalías con Isolation Forest - Alertas para operaciones atípicas

4. Integration - API síncrona: POST /api/v1/fraud/score - Timeout: 200ms - Fallback: Score = 50 (moderado) - Eventos asíncronos para actualización de modelos

8. Arquitectura de Autenticación y Autorización

8.1 Flujo de Autenticación - SPA Web

Flujo OAuth 2.1 / OpenID Connect (OIDC) Authorization Code con PKCE:



Paso 1: Usuario accede a SPA

Paso 2: SPA genera code_verifier (random string de 43-128 caracteres) y
 $\text{code_challenge} = \text{BASE64URL}(\text{SHA256}(\text{code_verifier}))$

Paso 3: SPA redirige a Keycloak con los parámetros de autorización incluyendo el code_challenge

Paso 4: Usuario ingresa credenciales en Keycloak

Paso 5: Keycloak valida credenciales y opcionalmente solicita MFA

Paso 6: Keycloak redirige de vuelta a SPA con el authorization code

Paso 7: SPA intercambia el authorization code por tokens, enviando el code_verifier

Paso 8: Keycloak valida el code_verifier contra el code_challenge y devuelve access_token, refresh_token e id_token

Paso 9: SPA almacena tokens en memoria (NO localStorage por seguridad XSS)

Paso 10: Todas las peticiones al API Gateway incluyen el access_token en el header
Authorization: Bearer {access_token}

Justificaciones:

1. **PKCE:** Previene authorization code interception attack
2. **Estado:** Previene CSRF attacks
3. **Tokens en memoria:** Previene XSS token theft
4. **Short-lived access token:** Limita ventana de compromiso
5. **Refresh token:** Permite renovación sin re-autenticación

8.2 Flujo de Autenticación - Aplicación Móvil

Flujo con Biometría Local:

1. Primera autenticación (igual que SPA con PKCE)
2. Refresh token se almacena en:
 - iOS: Keychain con kSecAttrAccessibleWhenUnlockedThisDeviceOnly
 - Android: EncryptedSharedPreferences con BiometricPrompt
3. Logins subsecuentes:
 - a. App solicita biometría (Face ID / Touch ID)
 - b. Si exitosa, desencripta refresh_token
 - c. Intercambia refresh_token por nuevo access_token:
POST /token
{
 grant_type: "refresh_token",
 refresh_token: {refresh_token}
}
 - d. Keycloak valida y emite nuevo access_token
4. App usa access_token para requests

Justificaciones:

1. **Biometría local:** Credenciales no salen del dispositivo
2. **Keychain/EncryptedSharedPreferences:** Hardware-backed encryption

3. **Refresh token rotation:** Cada uso genera nuevo refresh token (previene replay)

8.3 Flujo de Onboarding con Reconocimiento Facial

Proceso de Registro con Reconocimiento Facial:

Paso 1: Usuario inicia onboarding en app móvil

Paso 2: Captura de documento de identidad (cédula frontal y reverso)

Paso 3: AWS Textract extrae número de documento, nombres, apellidos, fecha de nacimiento y fecha de expedición

Paso 4: Validación contra Registro Civil del Ecuador (servicio gubernamental de validación de identidad) Civil del Ecuador

Paso 5: Captura de selfie con liveness detection - usuario sigue instrucciones como girar cabeza o parpadear

Paso 6: AWS Rekognition valida liveness con más del 95% de confianza

Paso 7: Face matching - compara selfie vs foto del documento con threshold del 98% de similitud

Paso 8: Validación contra listas restrictivas: OFAC, ONU, PEPs

Paso 9: Si todas las validaciones pasan, se crea usuario en Keycloak con credenciales temporales

Paso 10: Se envía email/SMS de confirmación con token de activación

Paso 11: Usuario activa cuenta y define contraseña

Paso 12: Registro de dispositivo con biometría local

Almacenamiento de Datos Biométricos: - NO se almacenan imágenes faciales (cumplimiento de la LOPDP) - Se almacena embedding facial (vector de 512 dimensiones) - Embedding encriptado con AES-256 - Usado solo para validaciones futuras de identidad

Justificaciones:

4. **Liveness detection:** Previene ataques con fotos o videos pregrabados
5. **Textract:** Automatiza extracción de datos, reduce errores humanos
6. **Alta similitud (98%):** Balance entre seguridad y UX

7. **Embeddings vs. imágenes:** Cumple con el principio de minimización de datos establecido en la LOPDP

8.4 Multi-Factor Authentication (MFA)

Escenarios de MFA Obligatorio:

1. **Primera transferencia a nuevo beneficiario**
2. **Transferencias > \$1000 USD**
3. **Cambio de contraseña**
4. **Transacción desde nuevo dispositivo/ubicación**
5. **Score de fraude > 70**
6. **Cambio de correo electrónico**
7. **Cambio de celular**
8. **Recuperación de credenciales**

Métodos de MFA Soportados:

1. **TOTP (Time-based One-Time Password):**
 - Google Authenticator, Authy
 - Código de 6 dígitos válido por 30 segundos
 - Algoritmo: RFC 6238
2. **SMS OTP:**
 - Código de 6 dígitos vía Twilio
 - Válido por 5 minutos
 - Máximo 3 intentos
 - Fallback si TOTP no disponible
3. **Biometría (Step-up Authentication):**
 - Re-solicitar Face ID/Touch ID en app móvil
 - Para confirmar transacciones críticas
 - No válido como único factor para transferencias grandes

Configuración en Keycloak:

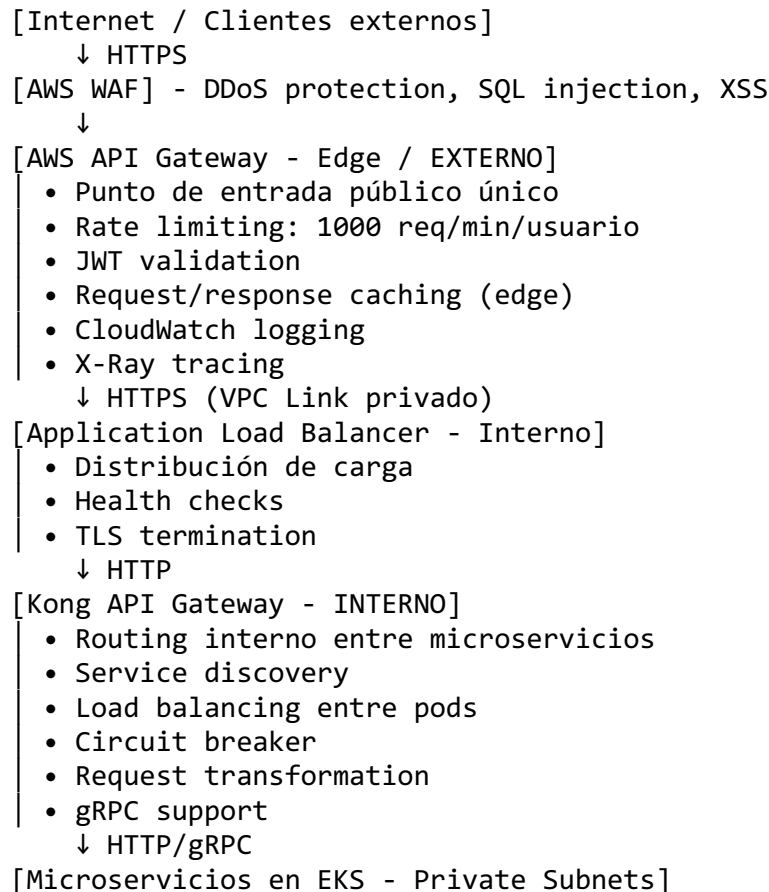
Authentication Flow: Browser

- └─ Cookie (ALTERNATIVE)
- └─ Identity Provider Redirector (ALTERNATIVE)
- └─ Forms (ALTERNATIVE)
 - └─ Username Password Form (REQUIRED)
 - └─ MFA Flow (CONDITIONAL)
 - └─ OTP Form (REQUIRED si configurado)
 - └─ WebAuthn (ALTERNATIVE)

9. Arquitectura de Integración y API Gateway

9.1 Capa de API Gateway

Arquitectura de Doble Gateway (Secuencial, NO al mismo nivel):



IMPORTANTE: Los gateways operan en capas diferentes de la arquitectura: - AWS API Gateway: Capa perimetral (Edge), cara pública al cliente - Kong Gateway: Capa interna (dentro del cluster Kubernetes), NO expuesto públicamente

Justificaciones del Doble Gateway:

- Separación de responsabilidades:**
 - AWS API Gateway: Seguridad perimetral, caché edge, protección DDoS
 - Kong: Routing interno, service mesh, comunicación inter-servicios
- Defensa en profundidad:**
 - WAF en edge (primera línea de defensa)
 - Validación en AWS API Gateway (segunda línea)
 - Kong valida nuevamente y aplica políticas internas (tercera línea)

- Network isolation con VPC (subnets privadas)
3. **Optimización:**
 - Caché en edge (CloudFront + API Gateway) para contenido estático
 - Caché en Kong para datos semi-estáticos
 - Reduce latencia en 40-60%
 4. **Flexibilidad:**
 - Kong permite gRPC para comunicación entre servicios internos
 - AWS API Gateway expone solo REST/HTTP hacia clientes externos
 - Cambios en routing interno no afectan contratos públicos

9.2 Patrones de Integración

Patrón 1: Strangler Fig para Core Bancario

Objetivo: Migración gradual del Core legacy sin Big Bang

Implementación:

1. Core Integration Service actúa como Anticorruption Layer
2. Routing basado en feature flags:
 - Si feature "new_balance_service" = ON → Nuevo microservicio
 - Si OFF → Core legacy
3. Migración incremental por funcionalidad
4. Dual writing durante período de transición
5. Validación de consistencia
6. Eventual reemplazo completo del Core

Justificaciones:

1. **Sin downtime:** Sistema funciona durante migración
2. **Rollback fácil:** Feature flag permite reversa inmediata
3. **Riesgo minimizado:** Migración por fases
4. **Testing en producción:** Canary deployment con % tráfico

Patrón 2: Circuit Breaker (Resilience4j)

```
@CircuitBreaker(name = "coreBanking", fallbackMethod = "fallbackGetBalance")
public Balance getBalance(String accountId) {
    return coreBankingClient.getBalance(accountId);
}

public Balance fallbackGetBalance(String accountId, Exception ex) {
    // Return cached balance or stale data with warning
```

```

        return cacheService.getLastKnownBalance(accountId)
            .orElseThrow(() -> new ServiceUnavailableException("Core temporal
mente no disponible"));
    }

```

// Configuración

```

circuitbreaker:
  configs:
    coreBanking:
      slidingWindowSize: 10
      failureRateThreshold: 50
      waitDurationInOpenState: 10s
      permittedNumberOfCallsInHalfOpenState: 3

```

Estados del Circuit Breaker: - **CLOSED:** Normal operation - **OPEN:** Fallo detectado, requests fallan inmediatamente con fallback - **HALF_OPEN:** Prueba si servicio se recuperó

Justificaciones:

1. **Protección de Core:** Evita cascade failures si Core se degrada
2. **Fast fail:** No esperar timeout si servicio está caído
3. **Auto-recovery:** Intenta reconectar automáticamente
4. **Graceful degradation:** Devuelve datos en caché

Patrón 3: Saga Pattern para Transferencias Distribuidas

Transferencia Interbancaria (Orchestration-based Saga):

Saga Coordinator (Transaction Service):

1. Inicio de saga
2. Comando: ValidateFraud → Fraud Service
 - ✓ Success → Continue
 - ✗ Failure → Abort saga
3. Comando: DebitAccount → Core Service
 - ✓ Success → Continue | Store compensation
 - ✗ Failure → Abort saga
4. Comando: CreditInterbankAccount → ACH Service
 - ✓ Success → Continue
 - ✗ Failure → Compensate: CreditAccount (reversa débito)
5. Comando: SendNotification → Notification Service
 - ✓ Success → Complete saga
 - ✗ Failure → Log error (no compensation, es informativo)
6. Saga completada exitosamente

Justificaciones:

1. **Consistencia eventual:** No hay transacciones distribuidas ACID

2. **Compensación automática:** Reversa automática en caso de fallo
3. **Idempotencia:** Comandos pueden reintentarse sin efectos secundarios
4. **Auditabilidad:** Cada paso es registrado

9.3 Protocolos de Comunicación

Externos (Cliente → API Gateway): - **HTTPS/TLS 1.3:** Encriptación en tránsito - **REST/JSON:** APIs públicas - **WebSocket (WSS):** Notificaciones en tiempo real (opcional)

Internos (Microservicio → Microservicio): - **HTTP/2 + REST:** Comunicación síncrona para consultas - **gRPC + Protocol Buffers:** Comunicación de alto rendimiento - 7x más rápido que REST/JSON - Usado para Transaction ↔ Fraud Detection - Typed contracts con .proto files - **Eventos (AWS EventBridge):** Comunicación asíncrona - Pub/sub para eventos de dominio - Desacoplamiento total entre servicios

Integración con Core Bancario: - **SOAP/XML:** Core legacy expone SOAP - **IBM MQ:** Mensajería para operaciones críticas - **Apache Camel:** Transformación SOAP ↔ REST

10. Arquitectura de Datos y Auditoría

10.1 Estrategia de Persistencia Poliglota

PostgreSQL - Datos Transaccionales

Uso: - Customer Service: Perfiles de usuario, consentimientos - Account Service: Metadatos de cuentas - Transaction Service: Registro de transferencias

Configuración AWS RDS:

Instancia: db.r6g.xlarge (4 vCPU, 32 GB RAM)

Storage: GP3 SSD, 500 GB, 12000 IOPS

Multi-AZ: Sí (zona primaria + standby)

Read Replicas: 2 réplicas para consultas

Backups:

- Automated daily snapshots (retención 7 días)
- Manual snapshots antes de deployments
- Point-in-time recovery (PITR)

Encryption:

- At rest: AES-256 con AWS KMS
- In transit: TLS 1.3

Particionamiento:

```
-- Partition de transactions por mes
CREATE TABLE transactions (
  id UUID PRIMARY KEY,
  user_id UUID NOT NULL,
  amount DECIMAL(18,2),
  created_at TIMESTAMP NOT NULL
) PARTITION BY RANGE (created_at);

CREATE TABLE transactions_2026_07 PARTITION OF transactions
  FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
-- ... más particiones
```

Índices Optimizados:

```
CREATE INDEX idx_transactions_user_date ON transactions(user_id, created_at DESC);
CREATE INDEX idx_transactions_status ON transactions(status) WHERE status != 'COMPLETED';
```

Redis ElastiCaché - Caché y Sesiones

Uso: - Sesiones de usuario (Keycloak session storage) - Caché de perfiles de cliente (TTL 5 min) - Caché de saldos (TTL 1 min) - Rate limiting counters - Distributed locks para transacciones

Configuración:

Node Type: cache.r6g.large (2 vCPU, 13.07 GB RAM)
Cluster Mode: Habilitado
Shards: 3 shards (distribución de carga)
Replicas: 2 réplicas por shard (HA)
Backup: Snapshot diario
Encryption: In-transit y at-rest

Patrón Caché-Aside Implementado:

```
async getCustomerProfile(userId: string): Promise<CustomerProfile> {
  // 1. Intenta obtener de caché
  const cacheKey = `customer:profile:${userId}`;
  const cached = await redis.get(cacheKey);

  if (cached) {
    this.metrics.increment('cache.hit');
    return JSON.parse(cached);
  }

  // 2. Caché miss - obtiene de DB
  this.metrics.increment('cache.miss');
  const profile = await this.customerRepository.findById(userId);
```

```

// 3. Almacena en caché
await redis.setex(cacheKey, 300, JSON.stringify(profile)); // TTL 5 min

return profile;
}

// Invalidación en updates
async updateCustomerProfile(userId: string, data: Partial<CustomerProfile>) {
  await this.customerRepository.update(userId, data);
  await redis.del(`customer:profile:${userId}`); // Invalidar caché
}

```

Justificaciones:

1. **Reducción de latencia:** De 50-100ms (DB) a <1ms (caché)
2. **Reducción de carga en DB:** 70-80% queries se resuelven en caché
3. **Escalabilidad:** Caché absorbe picos de tráfico

Amazon DocumentDB (compatible con MongoDB) - Base de Datos de Auditoría

Uso:

- Logs de auditoría inmutables
- Persistencia histórica de eventos para auditoría y compliance
- Logs de acceso a datos sensibles
- Trazabilidad completa

Configuración:

Cluster: 3 instancias (Primary + 2 Replicas)
Instance: db.r6g.large
Storage: Encrypted
Backup: Continuous backup con PITR
Retention: 7 años (compliance)

Schema de Auditoría:

```
{
  _id: ObjectId,
  eventType: "TRANSFER_EXECUTED",
  userId: "uuid",
  timestamp: ISODate("2026-07-22T10:30:00Z"),
  ipAddress: "190.xxx.xxx.xxx",
  deviceId: "device-uuid",
  action: {
    service: "TransactionService",
    method: "executeTransfer",
    parameters: {
      fromAccount: "****1234", // Masked
      toAccount: "****5678",
      amount: 300.00,
      currency: "USD"
    }
  },
  result: {
    status: "SUCCESS",
    transactionId: "txn-uuid",
    duration: 450 // ms
  },
  metadata: {
    userAgent: "BP-Mobile-iOS/2.1.0",
    location: {
      country: "EC",
      city: "Quito"
    }
  }
}

// Índices para queries rápidas
db.audit_logs.createIndex({ userId: 1, timestamp: -1 });
db.audit_logs.createIndex({ eventType: 1, timestamp: -1 });
db.audit_logs.createIndex({ "action.transactionId": 1 });

// TTL index para retención automática (7 años)
db.audit_logs.createIndex(
  { timestamp: 1 },
  { expireAfterSeconds: 220752000 } // 7 años en segundos
);
```

Justificaciones:

1. **Write-optimized:** Amazon DocumentDB maneja alto volumen de escrituras
2. **Schema flexible:** Diferentes tipos de eventos sin migraciones
3. **TTL automático:** Cumplimiento de retención sin procesos batch
4. **Queries rápidas:** Índices en campos clave

10.2 Patrón Event Sourcing para Auditoría

Implementación:

Cada cambio de estado se almacena como evento inmutable:

Ejemplo: Transferencia de \$100 USD

Event Stream:

1. TransferInitiated

```
{
  aggregateId: "txn-123",
  amount: 100,
  from: "acc-456",
  to: "acc-789",
  timestamp: "2026-07-22T10:30:00Z"
}
```
2. FraudCheckPassed

```
{
  aggregateId: "txn-123",
  score: 15,
  timestamp: "2026-07-22T10:30:01Z"
}
```
3. AccountDebited

```
{
  aggregateId: "txn-123",
  account: "acc-456",
  amount: 100,
  timestamp: "2026-07-22T10:30:02Z"
}
```
4. AccountCredited

```
{
  aggregateId: "txn-123",
  account: "acc-789",
  amount: 100,
  timestamp: "2026-07-22T10:30:03Z"
}
```

```
}
```

5. TransferCompleted

```
{  
  aggregateId: "txn-123",  
  timestamp: "2026-07-22T10:30:04Z"  
}
```

Estado actual = Replay de todos los eventos

Beneficios:

1. **Auditoría completa:** Cada paso es visible
2. **Time travel:** Reconstruir estado en cualquier punto del tiempo
3. **Debugging:** Reproducir errores exactamente
4. **Compliance:** Satisface requerimientos de auditoría financiera

10.3 Mecanismo de Persistencia para Clientes Frecuentes

Problema: Clientes VIP deben tener experiencia ultra-rápida

Patrón de Diseño Aplicado: Caché Warming (Predictive Caching)

```
// Background job que corre cada 5 minutos  
async warmupFrequentCustomersCachés() {  
  // 1. Identifica clientes frecuentes (> 10 Logins/semana)  
  const frequentCustomers = await this.analyticsService  
    .getFrequentCustomers(limit: 1000);  
  
  // 2. Pre-carga sus datos en Redis  
  for (const customer of frequentCustomers) {  
    const profile = await this.customerRepository.findById(customer.id);  
    const accounts = await this.accountRepository.findById(customer.id);  
    const recentTxns = await this.transactionRepository  
      .findRecent(customer.id, limit: 20);  
  
    // Almacenar en caché con TTL extendido  
    await redis.setex(`customer:profile:${customer.id}`, 600,  
      JSON.stringify(profile));  
    await redis.setex(`customer:accounts:${customer.id}`, 600,  
      JSON.stringify(accounts));  
    await redis.setex(`customer:recent-txns:${customer.id}`, 300,  
      JSON.stringify(recentTxns));  
  }  
  
  this.logger.info(`Caché warmed for ${frequentCustomers.length} customers`);  
}
```

Justificaciones:

1. **Proactividad:** Caché ya tiene datos antes de que usuario los pida
2. **Experiencia premium:** Clientes VIP tienen latencia < 50ms
3. **Reducción de carga:** DB no se golpea en momentos de alta demanda

Alternativa - Caché Aside con Read-Through:

- Evaluada pero descartada
 - Requiere que usuario “caliente” el caché con primer request
 - No cumple objetivo de experiencia premium desde primer acceso
-

11. Alta Disponibilidad y Recuperación ante Desastres

11.1 Objetivos de Disponibilidad

SLA: 99.95% uptime

Downtime permitido: 4.38 horas/año (26.3 min/mes)

RPO (Recovery Point Objective): < 5 minutos

RT0 (Recovery Time Objective): < 4 horas

11.2 Arquitectura Multi-Región

Configuración:

Región Primaria: us-east-1 (Virginia)

Región Secundaria: sa-east-1 (São Paulo)

Active-Active para lecturas

Active-Passive para escrituras (fallo automático)

Componentes por Región:

Cada región contiene:

- VPC con 3 AZs
 - Public Subnets (NAT Gateway, ALB)
 - Private Subnets (EKS nodes, microservicios)
 - Database Subnets (RDS, ElastiCaché)
- EKS Cluster
 - Node Groups (Auto Scaling)
 - Microservicios (Deployment + HPA)
 - Kong Gateway
- RDS PostgreSQL
 - Primary en AZ-1
 - Standby en AZ-2
 - Read Replica en AZ-3

- └─ ElastiCaché Redis
 - └─ 3 Shards con 2 replicas cada uno
- └─ Route 53 (DNS Failover)
- └─ S3 (Cross-Region Replication)

Replicación de Datos:

PostgreSQL:

- Primary en us-east-1
- Cross-region read replica en sa-east-1
- Replication lag: < 2 segundos
- Failover: Promoción automática de replica

ElastiCaché:

- Global Datastore (Redis 6.x+)
- Replicación cross-region en < 1 segundo
- Failover automático

S3:

- Cross-Region Replication (CRR) habilitada
- Replicación asíncrona
- Versioning habilitado

DocumentDB (Auditoría):

- Cluster en cada región
- Replicación eventual vía EventBridge

11.3 Estrategia de Failover

Detección de Fallo:

Route 53 Health Checks:

- └─ Endpoint: https://api.bp.com/health
- └─ Intervalo: 30 segundos
- └─ Failure threshold: 3 checks consecutivos
- └─ Timeout: 10 segundos
- └─ Evaluación: HTTP 200 + body "healthy"

Proceso de Failover Automático:

1. Route 53 detecta que región primaria no responde (90 segundos)
2. DNS actualizado para apuntar a región secundaria
3. RDS: Promoción de read replica a primary (60-120 segundos)
4. ElastiCaché: Failover automático a región secundaria (< 60 segundos)
5. EKS: Pods ya corriendo en región secundaria reciben tráfico
6. Notificación a equipo de operaciones (PagerDuty)
7. Total RTO: ~4 minutos para servicios críticos

TTL de DNS: 60 segundos (balance entre caché y failover rápido)

11.4 Auto-Scaling y Self-Healing

Horizontal Pod Autoscaler (HPA) - Kubernetes:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: transaction-service-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: transaction-service
  minReplicas: 3
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second
        target:
          type: AverageValue
          averageValue: "1000"
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Percent
          value: 50
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Pods
          value: 1
          periodSeconds: 60
```

Justificaciones:

1. **CPU + Memory + Custom Metrics:** Scaling basado en múltiples señales
2. **Scale-up rápido:** 50% de incremento para responder a picos
3. **Scale-down conservador:** Espera 5 min antes de reducir (evita flapping)
4. **Min 3 replicas:** Garantiza HA incluso en bajo tráfico

Self-Healing - Kubernetes Liveness/Readiness Probes:

```
livenessProbe:
  httpGet:
    path: /actuator/health/liveness
    port: 8080
  initialDelaySeconds: 60
  periodSeconds: 10
  timeoutSeconds: 5
  failureThreshold: 3

readinessProbe:
  httpGet:
    path: /actuator/health/readiness
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 5
  timeoutSeconds: 3
  failureThreshold: 2
```

Comportamiento: - **Liveness fail:** Kubernetes reinicia pod automáticamente -
Readiness fail: Kubernetes remueve pod de load balancer (no lo mata) - **Resultado:**
Pods degradados no reciben tráfico, sistema se auto-repara

11.5 Backup y Recuperación

Estrategia de Backups:

PostgreSQL RDS:

- Automated daily backups (3 AM UTC)
- Retention: 7 días
- Manual snapshots antes de deployments
- Point-in-time recovery: Hasta 5 minutos antes
- Cross-region snapshot copy a sa-east-1

ElastiCaché:

- Daily automatic backup (4 AM UTC)
- Retention: 3 días
- Manual backup antes de cambios críticos

DocumentDB:

- Continuous backup habilitado

- └─ Retention: 35 días
- └─ Point-in-time recovery

EKS (Persistent Volumes):

- └─ Velero para backup de PVs
- └─ Backup diario a S3
- └─ Retention: 30 días

Configuración (Infrastructure as Code):

- └─ Terraform state en S3 con versioning
- └─ Git repository con todos los configs
- └─ Automated deployment desde CI/CD

Procedimiento de Recuperación ante Desastre:

Escenario: Región completa caída (us-east-1)

- Paso 1: Failover de DNS (automático) - 2 min
- Paso 2: Promoción de RDS replica (automático) - 2 min
- Paso 3: Verificación de integridad de datos - 30 min
- Paso 4: Validación funcional de servicios críticos - 1 hora
- Paso 5: Comunicación a stakeholders - Inmediato
- Paso 6: Monitoreo intensivo - 24 horas

RTO Total: 3.5 horas (dentro del objetivo de 4 horas)

RPO: < 5 minutos (PITR + replicación continua)

12. Seguridad y Monitoreo

12.1 Modelo de Seguridad en Capas

Capa 1: Perímetro (Edge Security)

AWS Shield Standard: DDoS protection (Layer 3/4)

AWS WAF: Web Application Firewall

- └─ SQL Injection protection
- └─ XSS prevention
- └─ Rate limiting por IP (1000 req/5min)
- └─ Geo-blocking (bloqueo de países de alto riesgo)
- └─ Bot detection

CloudFront: CDN con TLS 1.3

- └─ Certificate: AWS Certificate Manager (ACM)
- └─ HTTPS enforcement (redirect HTTP → HTTPS)
- └─ Custom SSL certificate

Capa 2: Network Security

VPC Segmentation:

- Public Subnet (DMZ)
 - NAT Gateway
 - Application Load Balancer
 - Bastion Host (acceso SSH controlado)
- Private Subnet (Application Tier)
 - EKS Worker Nodes
 - Microservicios
 - Kong Gateway
- Database Subnet (Data Tier)
 - RDS
 - ElastiCaché
 - DocumentDB

Security Groups (Stateful Firewall):

- ALB-SG: Permite 443 desde 0.0.0.0/0
- EKS-SG: Permite tráfico desde ALB-SG
- DB-SG: Permite 5432 solo desde EKS-SG
- Deny all por defecto

Network ACLs (Stateless Firewall):

- Ingress: Allow 443, 80
- Egress: Allow all
- Backup de Security Groups

Capa 3: Application Security

API Gateway:

- JWT Validation (RS256)
- OAuth 2.1 / OpenID Connect (OIDC) scopes enforcement
- Request size limits (10 MB max)
- Response filtering (no headers internos)

Microservicios:

- mTLS entre servicios (Istio service mesh)
- RBAC (Role-Based Access Control)
- Input validation (Bean Validation)
- Output encoding (previene XSS)
- Secrets desde AWS Secrets Manager

Keycloak:

- Password policy:
 - Min 12 caracteres
 - Uppercase, lowercase, números, símbolos
 - No palabras comunes
 - Historial de 10 contraseñas
- Account lockout: 5 intentos fallidos

- └ Session timeout: 15 minutos de inactividad
 - └ Password expiry: Rotación basada en riesgo y lineamientos NIST 800-63
- B

Capa 4: Data Security

Encryption at Rest:

- └ RDS: AES-256 con AWS KMS
- └ EBS Volumes: Encrypted
- └ S3: Server-side encryption (SSE-KMS)
- └ ElastiCache: Encryption enabled
- └ Backups: Encrypted

Encryption in Transit:

- └ TLS 1.3 para clientes externos
- └ TLS 1.2+ para comunicación interna
- └ mTLS entre microservicios críticos

Data Masking:

- └ Números de cuenta: Últimos 4 dígitos visibles
- └ Emails: Parcialmente ofuscados
- └ Teléfonos: Últimos 4 dígitos visibles
- └ Logs: PII automáticamente redactado

Tokenization:

- └ Números de cuenta tokenizados con Vault
- └ Tokens no reversibles sin Vault
- └ Tokens con TTL configurable

12.2 Gestión de Secretos

HashiCorp Vault:

Vault en Kubernetes (HA):

- └ 3 replicas con Raft storage
- └ Auto-unseal con AWS KMS
- └ Dynamic secrets para DB credentials
- └ PKI para certificados internos
- └ Audit logging habilitado

Uso en microservicios:

- └ Vault Agent Injector (Kubernetes sidecar)
- └ Secrets inyectados como variables de entorno
- └ Auto-renovación de credentials
- └ Rotación automática cada 7 días

Alternativa evaluada: AWS Secrets Manager (más costoso, menos features)

12.3 Arquitectura de Monitoreo y Observabilidad

Tres Pilares de Observabilidad:

1. Metrics (Métricas)

Prometheus + Grafana:

Métricas de Negocio:

- transfers_total (counter)
- transfers_amount_total (counter)
- transfer_duration_seconds (histogram)
- fraud_score_distribution (histogram)
- active_users (gauge)

Métricas Técnicas:

- http_requests_total
- http_request_duration_seconds
- jvm_memory_used_bytes
- database_connections_active
- cache_hit_rate

Alertas:

- Error rate > 1% en 5 min → PagerDuty
- Latency P95 > 1s → Slack
- Database connections > 80% → Email
- Fraud score spike → SMS a equipo de seguridad
- RDS disk space < 20% → Ticket Jira

Dashboards Grafana:

- **Executive Dashboard:** KPIs de negocio (volumen, GMV, usuarios activos)
- **Operations Dashboard:** Health de servicios, latencias, error rates
- **Database Dashboard:** Conexiones, queries lentas, replication lag
- **Security Dashboard:** Intentos de login fallidos, rate limit hits, fraud scores

2. Logs (Registros)

ELK Stack (Elasticsearch, Logstash, Kibana):

Log Aggregation:

Microservicios → Filebeat → Logstash → Elasticsearch → Kibana

Structured Logging (JSON):

```
{
  "timestamp": "2026-07-22T10:30:00.123Z",
  "level": "INFO",
  "service": "transaction-service",
  "traceId": "abc123",
```

```

"spanId": "def456",
"userId": "user-789",
"message": "Transfer executed successfully",
"context": {
  "transactionId": "txn-xyz",
  "amount": 100,
  "duration": 450
}
}

```

Log Levels:

- ERROR: Failures que requieren investigación
- WARN: Situaciones anormales pero manejadas
- INFO: Eventos de negocio importantes
- DEBUG: Información detallada (solo dev/staging)
- TRACE: Información muy detallada (solo dev)

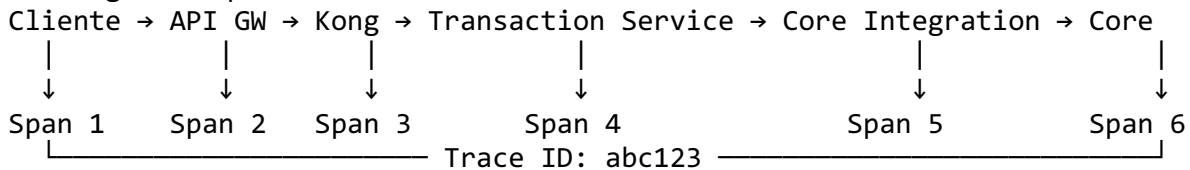
Retención:

- Hot tier (últimos 7 días): SSD rápido
- Warm tier (8-30 días): SSD estándar
- Cold tier (31-90 días): S3
- Delete después de 90 días (no compliance)

3. Traces (Trazas Distribuidas)

OpenTelemetry + AWS X-Ray:

Tracing de Request End-to-End:



Información capturada por span:

- Start time / End time
- Duration
- Service name
- HTTP method, path, status code
- Database queries ejecutadas
- Error details (si aplica)
- Custom tags (userId, accountId, etc.)

Sampling:

- 100% de requests con error
- 100% de requests > 2 segundos
- 10% de requests normales (reducir costo)
- Adaptive sampling basado en tráfico

Beneficios del Tracing: 1. **Debugging:** Identificar exactamente dónde falla un request 2. **Performance:** Ver qué servicio/query es lento 3. **Dependencies:** Mapa de dependencias entre servicios 4. **Compliance:** Trazabilidad completa para auditorías

12.4 Security Monitoring (SIEM)

AWS GuardDuty:

- Threat detection con ML
- Análisis de VPC Flow Logs
- Análisis de CloudTrail events
- Análisis de DNS logs
- Alertas a Security Hub

Amazon Detective:

- Investigación de incidentes
- Visualización de relaciones entre eventos
- Root cause analysis

Eventos monitoreados:

- Intentos de login fallidos consecutivos
- Acceso desde IPs sospechosas
- Queries SQL anormales
- Exportación masiva de datos
- Cambios en Security Groups
- Acceso a recursos desde IPs no reconocidas
- Patrones de escaneo de puertos

13. Estrategia de Cloud y Costos

13.1 Estimación de Costos Mensual (Producción)

Compute (EKS + EC2):

EKS Control Plane: $\$73/\text{cluster} \times 2 \text{ regiones} = \146

Worker Nodes (m6i.2xlarge):

- Producción: $12 \text{ nodos} \times \$0.384/\text{hora} \times 730\text{h} = \$3,365$
- Reserved Instances (1 año): $-40\% = \$2,019$
- Savings Plans adicional: $-10\% = \$1,817$

Total Compute: $\sim \$1,963/\text{mes}$

Databases:

RDS PostgreSQL (db.r6g.xlarge Multi-AZ):

- Instancia primaria: $\$0.52/\text{hora} \times 730\text{h} = \380
- Read replicas (2): $\$380 \times 2 = \760
- Storage (500 GB GP3): $\$115$
- Backup storage (500 GB): $\$50$
- Región secundaria (standby): $\$380$

ElastiCache Redis (cache.r6g.large):

- 3 shards × 3 nodos = 9 nodos
- $\$0.283/\text{hora} \times 9 \times 730\text{h} = \$1,859$

DocumentDB (db.r6g.large):

- 3 instancias × $\$0.29/\text{hora} \times 730\text{h} = \635
- Storage (1 TB): \$100

Total Databases: ~\$4,259/mes

Networking:

ALB: $\$22.50$ (base) + $\$0.008/\text{LCU} \times \text{estimado } 1000 \text{ LCU} = \30.50

NAT Gateway: $\$32.85/\text{mes} + \$0.045/\text{GB} \times 5\text{TB} = \257.85

CloudFront: $\$0.085/\text{GB} \times 10\text{TB} = \850

VPC Endpoints: $\$0.01/\text{hora} \times 5 \times 730\text{h} = \36.50

Data Transfer: ~\$500

Total Networking: ~\$1,675/mes

Storage:

S3 (documentos, backups, logs):

- Standard: $5\text{TB} \times \$0.023/\text{GB} = \118
- Intelligent Tiering: $10\text{TB} \times \$0.02/\text{GB} = \205
- Requests: ~\$50

EBS (EKS nodes): $12 \text{ nodes} \times 100\text{GB} \times \$0.10/\text{GB} = \$120$

Total Storage: ~\$493/mes

Security & Monitoring:

AWS WAF: $\$5$ (base) + $2 \text{ rules} \times \$1 = \7

GuardDuty: ~\$50

CloudWatch: Logs (50GB) + Metrics + Alarms = \$150

X-Ray: $1\text{M traces} \times \$5 \text{ per million} = \5

Secrets Manager: $100 \text{ secrets} \times \$0.40 = \$40$

Total Security & Monitoring: ~\$252/mes

Servicios Adicionales:

AWS Rekognition (onboarding):

- $10,000 \text{ face comparisons/mes} \times \$0.001 = \$10$
- Liveness detection: $5,000/\text{mes} \times \$0.025 = \$125$

SES (emails): $500\text{k emails} \times \$0.10/1000 = \$50$

SQS: $100\text{M requests} \times \$0.40/\text{M} = \$40$

EventBridge: $10\text{M events} \times \$1/\text{M} = \$10$

Lambda (auxiliar): ~\$30

Total Adicionales: ~\$265/mes

COSTO TOTAL MENSUAL: ~\$8,900/mes (~\$106,800/año)

Con optimizaciones (Reserved Instances, Savings Plans, rightsizing): COSTO OPTIMIZADO: ~\$7,100/mes (~\$85,200/año)

13.2 Estrategias de Optimización de Costos

1. Rightsizing de Instancias:

Análisis con AWS Compute Optimizer:

- Identificar instancias sobredimensionadas
- Recomendación: Reducir 20-30% de nodos en off-peak
- Ahorro: ~\$400/mes

2. Reserved Instances y Savings Plans:

Compromiso 1 año:

- 40% descuento en instancias base
- Ahorro: ~\$800/mes

Compromiso 3 años:

- 60% descuento
- Ahorro: ~\$1,200/mes (recomendado tras validación)

3. Auto-scaling Agresivo:

Configuración:

- Scale down agresivo en horarios nocturnos (11pm-6am)
- 50% reducción de capacidad
- Ahorro: ~\$300/mes

4. S3 Intelligent Tiering:

Migración automática a:

- Infrequent Access (30 días sin acceso)
- Archive (90 días sin acceso)
- Ahorro: ~\$150/mes

5. Database Optimization:

- Connection pooling (reduce RDS instances needed)
- Query optimization (reduce compute)
- Archive old data to S3 (reduce storage costs)
- Ahorro: ~\$200/mes

6. Spot Instances para Workloads No-Críticos:

Uso para:

- Batch jobs
- ML training
- Testing environments
- Ahorro: ~\$150/mes

Total Ahorro Anual: ~\$25,000 (23% reducción)

13.3 FinOps - Gobernanza de Costos

Tagging Strategy:

Recursos taggeados con:

```
Environment: production | staging | dev
Service: transaction-service | customer-service | ...
Team: payments | customer-experience | ...
CostCenter: engineering | operations
Project: internet-banking
```

Beneficios: - Cost allocation por equipo/servicio - Identificación de servicios más costosos - Chargeback a centros de costo - Optimización dirigida

Alertas de Presupuesto:

AWS Budgets:

- Alert a 50% del presupuesto mensual
- Alert a 75% del presupuesto
- Alert a 90% del presupuesto
- Notificaciones a CFO y CTO

14. Conclusiones

14.1 Resumen de Arquitectura

La solución propuesta para el sistema de banca por internet de BP Bank es una arquitectura moderna, escalable y segura que cumple con todos los requerimientos funcionales y no funcionales, así como con las regulaciones bancarias aplicables.

Fortalezas de la Arquitectura:

1. Escalabilidad y Performance:

- Arquitectura de microservicios permite escalamiento independiente
- Caché multi-nivel reduce latencia a < 200ms
- Auto-scaling garantiza capacidad en picos de demanda
- Target: 10,000 TPS con latencia P95 < 500ms

2. **Seguridad Robusta:**
 - Defensa en profundidad con múltiples capas de seguridad
 - OAuth 2.1 / OpenID Connect (OIDC) con PKCE para autenticación moderna
 - Biometría y MFA para operaciones críticas
 - Encriptación end-to-end (tránsito y reposo)
 - Cumplimiento PCI-DSS, LOPDP y regulaciones bancarias ecuatorianas
3. **Alta Disponibilidad:**
 - Multi-región active-passive con failover automático
 - 99.95% SLA (4.38h downtime/año)
 - Self-healing con Kubernetes
 - RPO < 5 minutos, RTO < 4 horas
4. **Observabilidad Completa:**
 - Métricas, logs y traces integrados
 - Alerting proactivo con PagerDuty
 - Dashboards ejecutivos y operacionales
 - SIEM para security monitoring
5. **Cumplimiento Normativo:**
 - Auditoría inmutable con event sourcing
 - Retención de 7 años automatizada
 - Validación contra listas restrictivas
 - KYC digital con reconocimiento facial
6. **Experiencia de Usuario:**
 - Aplicaciones nativas (React SPA + React Native)
 - Onboarding fluido con biometría
 - Notificaciones multi-canal en tiempo real
 - Autenticación passwordless con biometría

14.2 Roadmap de Implementación

Fase 1: Fundación (Meses 1-3) - Setup de infraestructura base en AWS - Implementación de Keycloak y autenticación - Desarrollo de API Gateway y Kong - Microservicios core: Customer, Account, Transaction - Integración básica con Core bancario

Fase 2: Funcionalidad Core (Meses 4-6) - Implementación de SPA React - Desarrollo de app móvil React Native - Integración con sistemas de notificación - Servicio de Fraud Detection básico - Auditoría y logging

Fase 3: Onboarding y Biometría (Meses 7-8) - Integración con AWS Rekognition - Flujo de onboarding completo - Autenticación biométrica en app móvil - Validación contra listas restrictivas

Fase 4: Optimización y HA (Meses 9-10) - Implementación de multi-región - Caché warming para clientes frecuentes - Optimización de performance - Testing de failover y DR

Fase 5: Production Hardening (Meses 11-12) - Penetration testing - Load testing (10k TPS) - Security audit y certificación PCI-DSS - Chaos engineering para validar resiliencia - Go-live progresivo (5% → 25% → 50% → 100% usuarios)

14.3 Riesgos y Mitigaciones

Riesgo 1: Dependencia de Core Bancario Legacy - **Mitigación:** Strangler Fig pattern, Circuit Breaker, fallback a caché - **Contingencia:** Microservicios pueden operar con funcionalidad reducida

Riesgo 2: Costos de Cloud Superiores a Estimado - **Mitigación:** Monitoreo continuo con AWS Cost Explorer, alertas de presupuesto - **Contingencia:** Plan de optimización agresiva, rightsizing

Riesgo 3: Latencia en Integraciones Externas (ACH) - **Mitigación:** Asincronía, patrón Saga, timeouts configurables - **Contingencia:** Transferencias quedan “pending” con notificación posterior

Riesgo 4: Fallo en Reconocimiento Facial (Falsos Negativos) - **Mitigación:** Threshold de similaridad configurable, proceso manual de revisión - **Contingencia:** Onboarding alternativo en sucursal física

Riesgo 5: Ataque de Seguridad / Breach - **Mitigación:** Múltiples capas de seguridad, WAF, IDS/IPS, GuardDuty - **Contingencia:** Plan de respuesta a incidentes, DR, comunicación a usuarios

Riesgo	Impacto	Mitigación
Indisponibilidad del Core Bancario	Alto	Circuit Breaker, Caché, Retry, Fallback
Falsos positivos del sistema antifraude	Medio	Revisión manual y ajuste periódico de modelos
Caída de proveedor biométrico	Medio	Proceso alternativo de validación y proveedor secundario
Saturación de ACH	Alto	Colas, reintentos e idempotencia
Incremento inesperado de tráfico	Alto	Auto Scaling en EKS y Redis
Dependencia de terceros (SMS, Email)	Medio	Estrategia multi-proveedor

14.4 Métricas de Éxito

Técnicas: - Uptime > 99.95% - Latency P95 < 500ms - Error rate < 0.1% - MTTR (Mean Time To Repair) < 30 minutos

Negocio: - 80% de transacciones por canal digital (vs. sucursales) - 90% satisfacción de usuarios (NPS) - 50% reducción en tiempo de onboarding (vs. proceso manual) - Zero multas regulatorias por incumplimiento

Seguridad: - Zero brechas de datos - 100% de transacciones auditadas - < 0.01% tasa de fraude

14.5 Sigüientes Pasos

1. **Aprobación de Arquitectura:** Revisión con stakeholders clave
2. **Proof of Concept:** Implementar un flujo end-to-end en 4 semanas
3. **Procurement:** Contratar servicios de AWS, licencias (si aplica)
4. **Team Formation:** Conformar equipos por dominio (payments, customer, security)
5. **Sprint 0:** Setup de repositorios, CI/CD, ambientes

Anexos

Anexo A: Tecnologías y Versiones

Frontend:

- React: 18.2.x
- TypeScript: 5.x
- React Native: 0.72.x
- Vite: 4.x
- Material-UI: 5.x

Backend:

- Node.js: 20 LTS
- NestJS: 10.x
- Java: 17 LTS
- Spring Boot: 3.1.x
- Python: 3.11

Infraestructura:

- Kubernetes: 1.27
- Keycloak: 24.x
- Kong: 3.4.x
- PostgreSQL: 15.x
- Redis: 7.x
- Amazon DocumentDB (compatible con MongoDB): 8.x
- Terraform: 1.5.x

Observabilidad:

- Prometheus: 2.45.x
- Grafana: 10.x
- Elasticsearch: 8.9.x
- Logstash: 8.9.x
- Kibana: 8.9.x
- OpenTelemetry: 1.x

Anexo B: Referencias y Estándares

Estándares de Seguridad: - OAuth 2.1 / OpenID Connect (OIDC): RFC 6749 - OAuth 2.1 / OpenID Connect (OIDC) Security Best Practices: RFC 8252 - PKCE: RFC 7636 - FAPI (Financial-grade API): OpenID Foundation - OWASP Top 10 2021 - PCI-DSS v4.0

Regulaciones: - Ley Orgánica de Protección de Datos Personales (LOPD) - Normativa de Seguridad de la Información y Gestión de Riesgos emitida por la Superintendencia de Bancos del Ecuador - ISO 27001:2013 - NIST Cybersecurity Framework

Patrones Arquitectónicos: - Microservices Patterns (Chris Richardson) - Domain-Driven Design (Eric Evans) - Enterprise Integration Patterns (Gregor Hohpe) - Building Microservices (Sam Newman)

Anexo C: Glosario

- **ACH:** Automated Clearing House - Red de pagos interbancarios
 - **ARCO:** Acceso, Rectificación, Cancelación, Oposición - Derechos de datos personales
 - **Circuit Breaker:** Patrón que previene cascade failures
 - **CQRS:** Command Query Responsibility Segregation
 - **DDoS:** Distributed Denial of Service attack
 - **Event Sourcing:** Patrón de persistencia basado en eventos inmutables
 - **FAPI:** Financial-grade API - Estándar de seguridad para APIs financieras
 - **KYC:** Know Your Customer - Proceso de identificación de clientes
 - **MFA:** Multi-Factor Authentication
 - **OFAC:** Office of Foreign Assets Control - Lista de sanciones de EE.UU.
 - **PEPs:** Politically Exposed Persons - Personas políticamente expuestas
 - **PKCE:** Proof Key for Code Exchange - Extensión de seguridad de OAuth
 - **RPO:** Recovery Point Objective - Pérdida máxima de datos aceptable
 - **RTO:** Recovery Time Objective - Tiempo máximo de recuperación
 - **Saga:** Patrón para transacciones distribuidas
 - **SLA:** Service Level Agreement - Acuerdo de nivel de servicio
 - **Strangler Fig:** Patrón de migración gradual de sistemas legacy
 - **TPS:** Transactions Per Second - Transacciones por segundo
 - **UAFE (Unidad de Análisis Financiero y Económico)**
-